Informe de Desarrollo: Parque Residencial, Sistema de Gestión de Parqueo

Introducción a la Programación y Computación 1, Segundo Semestre 2026 Universidad de San Carlos de Guatemala, Facultad de Ingeniería

Autor: Jonathan Enmanuel Gutierrez Estrada **Fecha:** 30 de octubre de 2026

1. Resumen

Este informe documenta el desarrollo del Sistema de Gestión de Parqueo para un residencial: el plan por etapas, el avance registrado, los problemas encontrados y las soluciones aplicadas. El sistema simula la llegada, permanencia y salida de vehículos usando estructuras de datos programadas desde cero (listas, colas y pila), cinco garitas que funcionan como hilos concurrentes y una interfaz Java Swing de seis paneles. La información de residentes y vehículos persiste en archivos de texto plano.

Resultado final: el sistema cumple el alcance obligatorio del enunciado. Se entrega como código en GitHub, un `.jar` ejecutable con archivos de ejemplo, manual técnico, manual de usuario y este informe.

2. Plan de implementación por etapas

Se siguió la metodología sugerida en el enunciado, implementando y probando cada pieza por separado antes de conectarla con la siguiente.

Etapa	Contenido	Prueba de aceptación
1. Análisis y diseño	Entidades, paquetes y diagrama de clases	Diagrama elaborado
2. Estructuras base	`NodoSimple`, `NodoDoble`, `ListaSimple`, `ListaDoble`, `Cola`, `Pila`, luego convertidas a genéricos	Pruebas por consola con salida esperada
3. Parqueo	150 espacios enlazados en cuatro direcciones, búsqueda, asignación y liberación	`contarEspacios()` devuelve 150 y los enlaces funcionan de ida y vuelta
4. Modelo y reglas	`Vehiculo`, `Residente`, `GestorResidentes`, máximo de 3 vehículos, regla de socios	`PruebaModelo` y `PruebaGestor`

Etapa	Contenido	Prueba de aceptación
5. Persistencia	Guardar y cargar `residentes.txt` y `vehiculos.txt`	Las listas se reconstruyen al reabrir
6. Hilos y sincronización	`ColaSegura`, `PilaSegura`, `GaritaEntrada`, `GaritaSalida`, `Simulacion`	10 eventos sin espacios repetidos; 12 eventos con entradas y salidas
7. Interfaz gráfica	Seis paneles Swing conectados a la simulación	Los cuadros cambian de color durante la simulación
8. Integración y pruebas	Casos límite desde la ventana	Los 12 casos de la tabla de pruebas del manual técnico
9. Documentación y entrega	Manuales, informe, `.jar`	`.jar` abre desde una carpeta limpia

3. Registro del avance

> Ajusta las fechas con `git log --date=short` para que coincidan exactamente con tus commits.

Fecha	Trabajo realizado
18 de octubre	Configuración del entorno (VS Code, JDK, Git), creación del repositorio y de los cinco paquetes
19 de octubre	Lista simple, lista doble, cola y pila; conversión a genéricos; parqueo de 150 espacios; `Vehiculo`, `Residente`, `GestorResidentes`; persistencia
20 de octubre	Concurrencia: `ColaSegura`, `PilaSegura`, garitas de entrada y salida, clase `Simulacion`
21 de octubre	Interfaz Swing con los seis paneles; guardado al cerrar y carga al iniciar; casos límite
28 de octubre	Limpieza, datos de ejemplo, `.jar`, manual técnico e informe

Se mantuvo un historial de commits incrementales con prefijos semánticos (`feat:`, `fix:`, `refactor:`, `chore:`), uno por cada pieza que funcionaba.

4. Problemas encontrados y soluciones aplicadas

4.1 `NullPointerException` en las listas

Problema: al insertar en una lista vacía, el código intentaba leer `actual.siguiente` cuando `actual` era `null`. **Solución:** se agregó un caso especial para la lista vacía. Se adoptó como regla de trabajo: antes de usar `algo.siguiente`, preguntarse si `algo` puede ser `null`. La misma regla se aplicó después a `buscarLibre`, `liberar`, `buscarPorId` y `cargar`.

4.2 Punteros de control mal actualizados

Problema: en la lista doble y en la cola, mover `cola` o `fin` antes de enlazar hace perder la referencia al último nodo; y en la cola, si `desencolar` vacía la estructura sin poner `fin = null`, el siguiente `encolar` engancha el nodo a uno que ya no existe. **Solución:** regla de **enlazar primero y mover el puntero después**, y mantener coherentes los dos punteros de control.

4.3 Conversión a genéricos a medias

Problema: al convertir las listas a `<T>`, el archivo `NodoDoble.java` quedó en la versión anterior con `Object`. El error `The type NodoDoble is not generic` aparecía en `ListaDoble` y generaba decenas de errores derivados. Además, Git indicaba "nothing to commit" porque los cambios no estaban guardados en disco. **Solución:** se arregló la causa común (el nodo) y los demás errores desaparecieron. Se adoptó el hábito de guardar todos los archivos (`Ctrl+K S`) antes de compilar y de hacer commit.

4.4 Código pegado en el archivo equivocado

Problema: el código de `Pila` se pegó dentro de `PilaSegura.java`, lo que produjo 14 errores ("declared package does not match" y "must be defined in its own file"). En otras ocasiones, métodos quedaron fuera de la clase por una llave de más (`GestorResidentes`, `Simulacion`) o el bloque de un hilo quedó sin cerrar (`}).start();`). **Solución:** reemplazar el archivo completo en lugar de buscar llaves, formatear con `Shift+Alt+F` para ver si el método queda dentro de la clase, y verificar siempre el nombre de la pestaña antes de pegar. Lección: ante muchos errores de sintaxis a la vez, arreglar primero el de más arriba.

4.5 Espacios en blanco sin completar

Problema: un hueco `______` del código guiado quedó escrito literalmente y el compilador lo trató como un método inexistente. **Solución:** reemplazarlo por `obtenerEn(j)`. Lección: un error con guiones bajos casi siempre es un hueco sin completar.

4.6 Pruebas desactualizadas tras cambiar una firma

Problema: al cambiar `asignar(String placa, ...)` por `asignar(Vehiculo v, ...)`, las pruebas anteriores dejaron de compilar. Algo similar ocurrió al quitar el parámetro `esSocio` del constructor de `GaritaEntrada`. **Solución:** actualizar las pruebas. Un cambio de tipo debe propagarse a todo el código que lo usa; el IDE marca cada lugar pendiente.

4.7 Condición de carrera en la asignación de espacios

Problema: `asignar` busca un espacio libre y después lo ocupa. Con tres garitas en paralelo, dos podrían ver el mismo espacio libre y la segunda sobrescribir a la primera. **Solución:** los métodos del parqueo que leen y modifican el estado son `synchronized`; `asignarSegunTipo` es un solo bloque protegido. Para entender el fenómeno se construyó `PruebaCarrera`, donde `contador++` sin sincronizar da menos de 200 000 por esconder tres pasos (leer, sumar, escribir).

4.8 Garitas procesando una cola vacía

Problema: si `wait()` estuviera dentro de un `if`, una garita despertada por `notifyAll()` podría sacar de una cola que otra garita ya vació, y obtener `null`. **Solución:** `wait()` dentro de un `while`, de modo que cada garita vuelva a verificar la condición al despertar.

4.9 La ventana mostraba un parqueo distinto al de las garitas

Problema: `VentanaPrincipal` creaba su propio `new Parqueo()`, distinto del que usaban las garitas dentro de `Simulacion`. La interfaz nunca mostraba un vehículo estacionado. **Solución:** la ventana recibe la `Simulacion` y usa `sim.getParqueo()`, de modo que todos comparten el mismo objeto.

4.10 Todos los vehículos caían en el área de socios

Problema: el valor `esSocio` estaba fijo en `true` durante las pruebas del hilo, y quedó así al integrar. Cualquier vehículo, socio o no, ocupaba primero el área A-C, violando la regla del enunciado. **Solución:** `GaritaEntrada` recibe el gestor y calcula `esSocio` a partir del dueño del vehículo (`buscarDuenoPorPlaca`). Los visitantes, que no tienen dueño, solo reciben espacios del área general.

4.11 Misma placa ocupando varios espacios

Problema: pulsar "Llega vehículo" dos veces con la misma placa ocupaba dos espacios, pero "Pedir salida" liberaba solo uno; los espacios se acumulaban como ocupados. **Solución:** se agregó `Parqueo.estaEstacionada(placa)` y el panel de vehículos rechaza la llegada si la placa ya está estacionada.

4.12 La cuadrícula no cabía en la ventana

Problema: con seis pestañas, las 25 columnas del área de socios y las cinco filas del área general no cabían. **Solución:** el panel de parqueo se envolvió en un `JScrollPane` y se amplió la ventana.

4.13 Variable declarada dos veces

Problema: al agregar la carga de datos al `main`, la variable `sim` quedó declarada dos veces (`Duplicate local variable sim`). **Solución:** eliminar la declaración antigua y dejar una sola, creada después de cargar el gestor.

5. Decisiones tomadas durante el desarrollo

- **Regla de socios:** el socio intenta A-C y, si está llena, usa el área general; no socios y visitantes solo usan el área general.
- **Estructuras genéricas:** se prefirió `<T>` sobre `Object` para evitar conversiones y detectar errores al compilar.
- **Pantalla:** se actualiza con un `Timer` de 500 ms que lee el estado sin bloquearlo.
- **Visitantes:** solo viven en memoria; no se agregan al gestor y por eso no se persisten.
- **Tipo de vehículo al cargar:** `"carro"` por defecto, porque el formato sugerido no lo incluye.

6. Aprendizajes

- Los nodos y las referencias son la base de todas las estructuras; el patrón de recorrido `while (actual != null)` se reutiliza en todo el proyecto.
- Una estructura con dos punteros de control exige mantenerlos coherentes.
- La concurrencia exige pensar qué pasos son compuestos: lo que parece una operación puede ser "buscar y luego ocupar".
- `wait()` va en `while`, y `volatile` garantiza visibilidad pero no atomicidad.
- Swing no es seguro para hilos: las garitas nunca tocan componentes gráficos; la interfaz consulta su estado.

- Leer los errores del IDE ayuda a encontrar la causa común antes de corregir cada línea roja.
- Commits pequeños y frecuentes facilitan volver a un estado que funcionaba.

7. Estado final frente a la rúbrica

Criterio	Estado
Interfaz gráfica (seis paneles)	Completo
Simulación del flujo para residentes, socios y visitantes	Completo
Listas enlazadas (residentes y vehículos)	Completo, con máximo de 3 vehículos
Lista de listas del parqueo (150 espacios)	Completo, enlazado en las cuatro direcciones
Colas y pila	Completo, integradas con la simulación y la interfaz
Hilos y sincronización	Cinco hilos, `synchronized`, `wait()` en `while` y `notifyAll()`
Persistencia	Guarda al cerrar y reconstruye al iniciar; visitantes no se guardan
Documentación	Manual técnico, manual de usuario e informe

No se implementó el alcance opcional (deshacer, pausar y ajustar velocidad, llegadas automáticas).